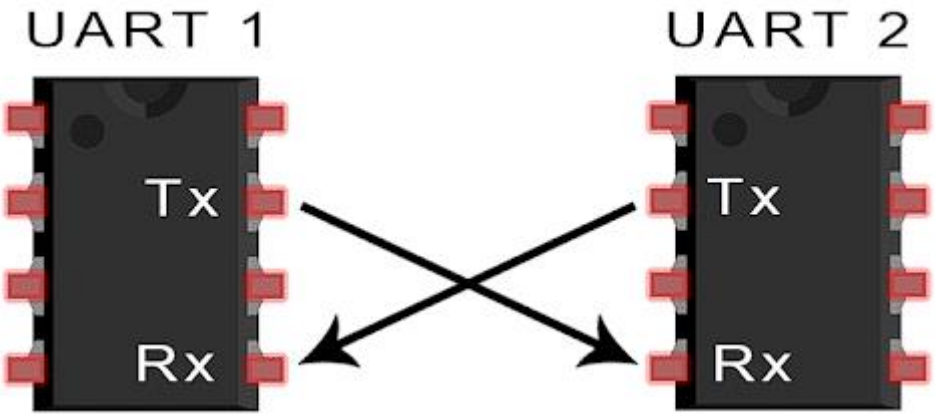
Лабораторная работа №8.
Приём и передача данных по протоколу UART в STM32

Цель работы: научиться осуществлять передачу и приём данных по интерфейсу UART в STM32, используя библиотеку HAL.

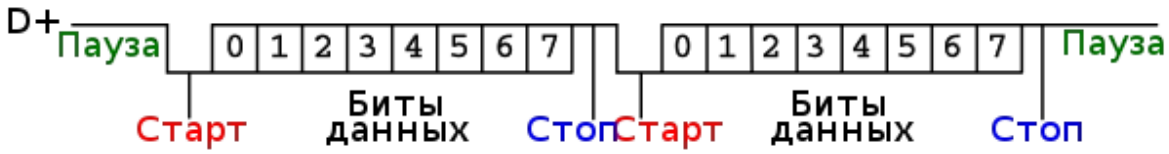
UART

UART (Universal Asynchronous Receiver-Transmitter), что в переводе звучит как Универсальный Асинхронный Приемо-Передатчик. “Универсальный” значит широко распространен в электронном мире. Его в разных видах используют и большие и малые компьютеры, контроллеры, датчики, средства коммуникации и прочие электронные устройства. Слово “асинхронный” означает то, что прием и передача отдельных битов не выравниваются так называемыми синхроимпульсами, что и хорошо, и плохо одновременно. Единица отличается от нуля исключительно по времени между перепадами уровней сигнала, которое заранее задается скоростью передачи.

С аппаратной стороны UART использует два пина контроллера - RX и TX, где первые буквы обозначают, соответственно, Receiver и Transmitter. Для связи двух устройств понадобятся два провода, причем соединять их следует крест-накрест RX первого в TX второго и наоборот. Куда один передает, там другой принимает.



С программной стороны, в общем случае, передача выглядит как цепочка сигналов, а именно битов, разделенных на байты, плюс (опционально) сигналов служебных.



Пока информация не передается, в линии сохраняется высокий уровень. Спад сигнала - это команда принимающей стороне, что сейчас придет информация. И спустя определенное количество времени, зависящее от заранее заданной скорости, начинается передача байта в виде ряда нулей и/или единиц с соблюдением тех же, заранее оговоренных, временных отрезков. После восьмого бита следует стоп-сигнал в виде высокого уровня и ситуация повторяется до тех пор, пока не будут переданы все нужные нам байты.

Главное, что надо знать о работе UART:

- в неактивном режиме (состоянии ожидания) выход передатчика UART находится в высоком состоянии;
- передача байта начинается со стартового бита, который равен 0 (низкий уровень сигнала);
- передача байта заканчивается стоповым битом, который равен 1 (высокий уровень сигнала);
- данные передаются, начиная с младшего бита;
- для передачи байта требуется 10 битов;
- время передачи одного байта вычисляется исходя из скорости обмена и количества битов в пакете (10 бит при передаче байта).

UART или USART

Вы часто будете встречать обе эти аббревиатуры. В чем разница?

- **UART**: **U**niversal **A**synchronous **R**eceiver-**T**ransmitter (Универсальный **Асинхронный** Приёмопередатчик).
- **USART**: **U**niversal **S**ynchronous/**A**synchronous **R**eceiver-**T**ransmitter (Универсальный **Синхронно-Асинхронный** Приёмопередатчик).

Ключевое слово — **Синхронный**.

Асинхронный режим (UART) — это связь без отдельного тактового сигнала. Представьте двух людей, договорившихся говорить со скоростью ровно 120 слов в минуту. Им не нужен дирижер (тактовый сигнал), потому что они оба *заранее договорились* о скорости. Если один начнет говорить быстрее или медленнее, другой его не поймет. В этом режиме используются всего два провода: **TX** (Transmit) и **RX** (Receive).

Синхронный режим (USART) — это связь с тактовым сигналом. Один из участников (Master) генерирует на отдельном проводе **CK** (Clock) тактовые импульсы, а второй (Slave) читает и пишет данные строго по этим импульсам. Это надежнее, позволяет работать на более высоких скоростях и не боится небольших расхождений в "скорости" устройств.

Важный факт о UART в STM32: Почти все модули, которые вы будете использовать в STM32, называются USART. Однако в 99% случаев вы будете конфигурировать их для работы в **асинхронном режиме**, то есть использовать их как обычный UART. Поэтому далее мы будем говорить в основном об асинхронном режиме.

Регистры STM32

USART_SR

USART_SR - регистр состояния.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Прежде всего, нам интересны флаги, сообщающие о состоянии приема и передачи.

- бит 7 TXE. Буферный регистр передатчика пуст. Флаг становится равным 1 после того, как данное перегружается в регистр сдвига. Т.е. флаг сообщает о том, что буферный регистр пуст, может быть загружено новое данное. Флаг устанавливается аппаратно и сбрасывается после записи байта в буферный регистр USART_DR.
- бит 6 TC. Флаг устанавливается аппаратно и сообщает о том, что передача данного закончена, сдвиговый регистр пуст. Если предыдущий флаг (TXE) говорит о том, что можно в буферный регистр загружать новое данное. Физическая передача может продолжаться, на выходе TX возможно идет поток битов. То флаг TC сообщает, что все биты переданы. Часто этот момент необходимо определять для отключения передатчика шинного интерфейса сети, например, в интерфейсе RS-485. Сбрасывается флаг последовательным чтением регистров USART_SR и USART_DR.
- бит 5 RXNE. Буферный регистр приема не пуст. Флаг сообщает, что в буферном регистре приемника есть данное. Данное должно быть считано до прихода следующего, иначе оно будет потеряно. Сбрасывается флаг при чтении регистра USART_DR.

UART способен определить некоторые ошибки приема данных, возникающие из-за искажения сигнала при передаче или за счет неправильных программных операций с ним. Об ошибках сигнализируют следующие 4 бита.

- Бит 3 ORE. Ошибка переполнения. Флаг устанавливается в случае, если в приемный буферный регистр поступило новое данное, а предыдущее считано не было. Т.е. при потере данного;
- Бит 2 NE. Флаг устанавливается при выделении шума во входном сигнале. Наличие шума определяется как слишком частое переключение входного сигнала;
- Бит 1 FE. Ошибка приема фрейма (кадра). Возникает, когда не был выделен стоп-бит. Т.е. после приема стартового бита и 8 битов данных на месте стопового бита UART считал сигнал низкого уровня.
- Бит 0 PE. Ошибка паритета. Сигнализирует об ошибке при включенном контроле четности;
- Все биты сбрасываются программно последовательным чтением сначала регистра USART_SR, затем USART_DR.

USART_DR

USART_DR - регистр данных.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							DR[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw

Используется для записи данных в буферный регистр передатчика и чтения данных из буферного регистра приемника. На самом деле это 2 регистра, для обращения к которым используется один адрес.

USART_BRR

USART_BRR - регистр, задающий скорость обмена.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр содержит значение делителя частоты (USARTDIV), который определяет скорость передачи и приема данных.

Скорость вычисляется по формуле:

BAUD = Fck / (16 * USARTDIV), где

- BAUD – скорость, бод.
- Fck – входная частота тактирования UART:
- сигнал PCLK2 для UART1 (шина APB2);
- сигнал PCLK1 для UART2 и 3 (шина APB1).
- USARTDIV – значение регистра USART_BRR.

USARTDIV задано в формате с фиксированной запятой с дробной частью.

USART_CR1

USART_CR1 – управляющий регистр 1.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Бит 13 UE. Разрешение работы UART. Отключение UART (UE=0) уменьшает ток потребления микроконтроллера.
- Бит 12 M. Задает длину слова 8 бит (M=0) или 9 бит (M=1).
- Бит 10 PCE. Разрешение контроля четности (PCE=1).
- Бит 7 TXEIE. Разрешение прерывания по флагу TXE, т.е. когда буферный регистр передатчика пуст.
- Бит 6 TCIE. Разрешение прерывания по флагу TC, т.е. когда пуст сдвиговый регистр передатчика.
- Бит 5 RXNEIE. Разрешение прерывания по флагу RXNE, т.е. когда в буферном регистре приемника есть непрочитанное данное.
- Бит 3 TE. Разрешение работы передатчика.
- Бит 2 RE. Разрешение работы приемника.

USART_CR2

USART_CR2 – управляющий регистр 2.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LINEN	STOP[1:0]		CLK EN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	Res.	ADD[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Нам интересно только одно поле. Биты 13:12 STOP. Биты задают формат стопового признака слова:

- 00 – 1 стоп-бит;
- 01 – 0,5 стоп-бита;
- 10 – 2 стоп-бита;
- 11 – 1,5 стоп бита.

HAL_UART_HandleTypeDef

Структура HAL_UART_HandleTypeDef используется в STM32 HAL (Hardware Abstraction Layer) для управления и настройки UART (Universal Asynchronous Receiver/Transmitter). Она содержит все необходимые поля для конфигурации UART, управления состоянием и обработки данных. Ниже приведено подробное описание этой структуры.

```
typedef struct __UART_HandleTypeDef
{
    USART_TypeDef          *Instance;      /* Указатель на регистры UART
    (например, USART1, USART2 и т.д.) */

    UART_InitTypeDef       Init;           /* Конфигурация UART (скорость, биты
    данных, стоп-биты и т.д.) */
    uint8_t                *pTxBuffPtr;    /* Указатель на буфер передачи */
    uint16_t               TxXferSize;     /* Размер данных для передачи */
    uint16_t               TxXferCount;    /* Оставшееся количество данных для
    передачи */
    uint8_t                *pRxBuffPtr;    /* Указатель на буфер приема */
    uint16_t               RxXferSize;     /* Размер данных для приема */
    uint16_t               RxXferCount;    /* Оставшееся количество данных для
    приема */
    DMA_HandleTypeDef      *hdmatx;        /* Указатель на структуру DMA для
    передачи */
    DMA_HandleTypeDef      *hdmarx;        /* Указатель на структуру DMA для
    приема */
    HAL_LockTypeDef        Lock;           /* Объект блокировки для обеспечения
    потокобезопасности */
    __IO HAL_UART_StateTypeDef  State;     /* Текущее состояние UART */
    __IO uint32_t          ErrorCode;      /* Код ошибки UART */
} UART_HandleTypeDef;
```

Instance:

Указатель на структуру регистров UART (например, USART1, USART2 и т.д.). Это поле связывает структуру с конкретным аппаратным модулем UART.

Init:

Структура типа UART_InitTypeDef, которая содержит параметры инициализации UART:

- BaudRate: Скорость передачи (например, 9600, 115200 и т.д.).
- WordLength: Длина слова (8 бит, 9 бит и т.д.).
- StopBits: Количество стоп-битов (1, 2 и т.д.).
- Parity: Контроль четности (None, Even, Odd).
- Mode: Режим работы (передача, прием или оба).
- HwFlowCtl: Управление потоком (None, RTS, CTS, RTS/CTS).
- OverSampling: Передискретизация (8x или 16x).

pTxBuffPtr:

Указатель на буфер данных, которые нужно передать.

TxXferSize:

Общий размер данных для передачи (в байтах).

TxXferCount:

Оставшееся количество байт для передачи. Уменьшается по мере передачи данных.

pRxBuffPtr:

Указатель на буфер для приема данных.

RxXferSize:

Общий размер данных для приема (в байтах).

RxXferCount:

Оставшееся количество байт для приема. Уменьшается по мере приема данных.

hdmatx:

Указатель на структуру DMA для передачи данных. Используется, если передача данных осуществляется через DMA.

hdmarx:

Указатель на структуру DMA для приема данных. Используется, если прием данных осуществляется через DMA.

Lock:

Объект блокировки, используемый для обеспечения потокобезопасности при работе с UART. Предотвращает одновременный доступ к ресурсам UART из разных потоков.

State: Текущее состояние UART. Может принимать значения из перечисления HAL_UART_StateTypeDef, например:

- HAL_UART_STATE_RESET: UART в состоянии сброса.
- HAL_UART_STATE_READY: UART готов к работе.
- HAL_UART_STATE_BUSY: UART занят (например, выполняется передача или прием).
- HAL_UART_STATE_BUSY_TX: UART занят передачей данных.
- HAL_UART_STATE_BUSY_RX: UART занят приемом данных.
- HAL_UART_STATE_BUSY_TX_RX: UART занят одновременно передачей и приемом.

ErrorCode: Код ошибки UART. Может содержать комбинацию флагов ошибок, таких как:

- HAL_UART_ERROR_NONE: Ошибок нет.
- HAL_UART_ERROR_PE: Ошибка четности.
- HAL_UART_ERROR_NE: Шумовая ошибка.
- HAL_UART_ERROR_FE: Ошибка фрейма.
- HAL_UART_ERROR_ORE: Ошибка переполнения.
- HAL_UART_ERROR_DMA: Ошибка DMA.

HAL_UART_Init

*HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart);*

Инициализирует UART со всеми установленными параметрами.

huart - указатель на структуру *UART_HandleTypeDef,* содержащую конфигурацию UART.

Возвращаемое значение: статус HAL (HAL_OK, HAL_ERROR).

HAL_UART_Transmit

*HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)*

Отправляет байты в блокирующем режиме (блокирующий значит, что никакие другие операции на передачу/прием не будут выполняться)

Параметры:

- huart - указатель на структуру *UART_HandleTypeDef,* содержащую конфигурацию UART;
- pData - указатель на данные, которые будут отправлены (тип данных - 1 байт);
- Size - размер передаваемых данных;
- Timeout - таймаут выполнения функции. Если функция не будет выполнена в течение, например, 100 мс таймаута, функция вернет статус HAL_TIMEOUT.

Возвращаемое значение: статус *HAL* (*HAL_OK*, *HAL_ERROR*, *HAL_TIMEOUT*).

HAL_UART_Receive

*HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)*

Получает количество заданных байт в блокирующем режиме

Параметры:

- huart - указатель на структуру *UART_HandleTypeDef*, содержащую конфигурацию UART;
- pData - указатель на массив полученных данных (тип данных - 1 байт);
- Size - размер получаемых данных;
- Timeout - таймаут выполнения функции. Если функция не будет выполнена в течение, например, 100 мс таймаута, функция вернет статус HAL_TIMEOUT.

Возвращаемое значение: статус *HAL* (*HAL_OK*, *HAL_ERROR*, *HAL_TIMEOUT*).

Нюансы и частые ошибки

1. **Baud Rate Mismatch (Несовпадение скоростей)**. Самая частая ошибка. Вы настроили STM32 на 115200, а терминал на ПК на 9600. Результат — нечитаемые символы ``.
2. **GPIO Configuration (Настройка пинов)**. Пины TX и RX должны быть настроены в режим **Alternate Function (AF)**. Если они останутся в режиме GPIO_Output или GPIO_Input, модуль UART не будет к ним подключен. CubeMX делает это за вас, но помнить об этом нужно.
3. **Общая земля (GND)**. Для передачи сигнала (разности потенциалов) нужен общий "ноль". Если вы соединяете два устройства (например, STM32 и GPS-модуль) только проводами TX и RX, *ничего работать не будет*. Обязательно соедините их выводы GND.
4. **Логические уровни**:
   - **UART (TTL)**: Используется в микроконтроллерах. Логическая '1' = 3.3V (или 5V), '0' = 0V.
   - **RS-232**: Старый стандарт для COM-портов ПК. '1' = от −3V до −25V, '0' = от +3V до +25V.
   - **НИКОГДА** не подключайте пины STM32 (3.3V) напрямую к COM-порту ПК (+/−12V). Вы сожжете микроконтроллер. Используйте USB-to-UART (TTL) адаптеры (FTDI, CP2102, CH340) или микросхему-преобразователь уровней (например, MAX3232).
5. **TXE vs. TC**:
   - TXE (Transmit Empty) = **буфер** пуст. Можно слать *следующий* байт.
   - TC (Transmission Complete) = **сдвиговый регистр** пуст. *Последний* байт *полностью* ушел с ножки. Это важно для интерфейсов вроде RS-485, где вам нужно переключать приёмопередатчик из режима TX в RX ровно после того, как ушел последний бит. Для этого нужно ждать флага TC, а не TXE.
6. **Overrun Error (Ошибка переполнения)**. Аппаратура выставила флаг RXNE (данные пришли), а ваш CPU не успел их прочитать из DR до того, как пришел *следующий* байт. Старый байт в DR будет затерт новым. Флаг ORE будет выставлен. Это знак, что ваш код слишком медленный. Лечение — использовать IT или DMA.

Задание:

1. Реализуйте передачу строки “Hello World!” на ПК по интерфейсу UART. Сообщение должно передаваться каждую секунду.

   В качестве программы для работы с UART можете выбрать Terminal, Putty или любую другую программу для работы с UART-интерфейсом по COM-порту.
   Для секундной задержки используйте функцию HAL_Delay.

2. Реализуйте передачу строки “Button is hold!” на ПК по интерфейсу UART. Сообщение должно передаваться по нажатию кнопки, расположенной на вашей отладочной плате.
   Имейте в виду, что при нажатии кнопки передача строки может выполниться многократно. Поэтому лучше реализовать передачу строки по внешнему прерыванию после нажатия кнопки.

3. Реализуйте прием строки “Hello from PC!” на стороне STM32. Строку должен передать ваш ПК.
   Для передачи строки вы также можете воспользоваться программой Terminal, Putty или любой другой программу для работы с UART-интерфейсом по COM-порту.

4. *Необязательно* (при наличии ЦАП (DAC) на плате). Реализуйте передачу с ПК на STM32 одной из строк:
   - “Triangle”;
   - “Sine”;
   - “PWM”.

   В зависимости от пришедшей строки, реализуйте генерацию сигнала с ЦАП:
   - если “Triangle” – должен генерироваться треугольный сигнал;
   - если “Sine” – должен генерироваться синусоидальный сигнал;
   - если “PWM” – должен генерироваться ШИМ-сигнал.

   Характеристики генерируемого с ЦАП сигнала выберите на своё усмотрение!

5. Представьте проделанные шаги 1-3 (дополнительно 4) в отчёте. Покажите в отчёте, какие строки вы отправляете и какие строки вы получаете (на ПК и на STM32).
   Также приложите в ответе на задание файл(-ы) main.c с выполнением заданий.